King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department

CPCS 211: Digital Logic Design

Chapter 5 : Designing Compinational Systems

Part II (5.4 – 5.6 & 5.8)

Outline

- Multiplexers
- DeMultiplexers
- Three State Gates and Bus
- Gate Arrays: ROMs, PLAs and PALs
- Designing with Read Only Memories
- Larger Examples of Combinational Circuits

Multiplexer

- A multiplexer, often referred as a mux, is basically a switch that passes one of its data inputs through to the output, as a function of a set of select inputs

- A multiplexer is a combinational circuit that selects binary information from one of many input lines and directs it to a single output line

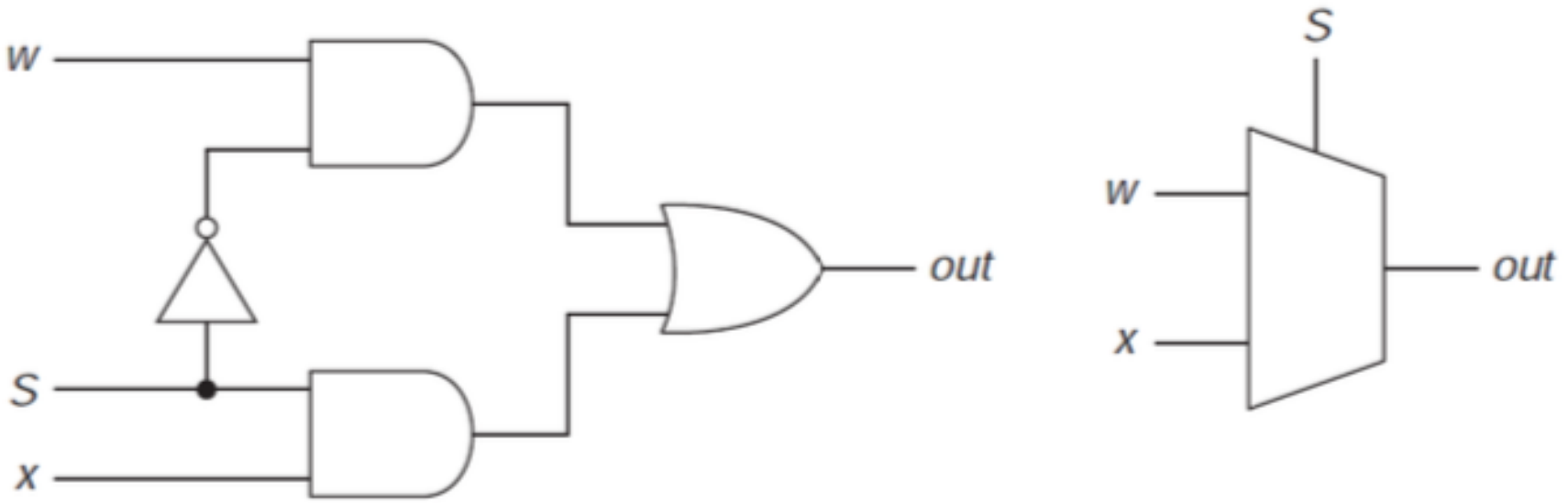
Multiplexer

- Selection of a particular input line is controlled by a set of selection lines
- Normally there are 2^n input lines and n selection lines whose bit
- combinations determine which input is selected
- A two-to-one line multiplexer connects one of two 1-bit sources to a common destination, as shown next

Multiplexer

A two-way multiplexer and its logic symbol are shown

Figure 5.11 Two-way multiplexer.



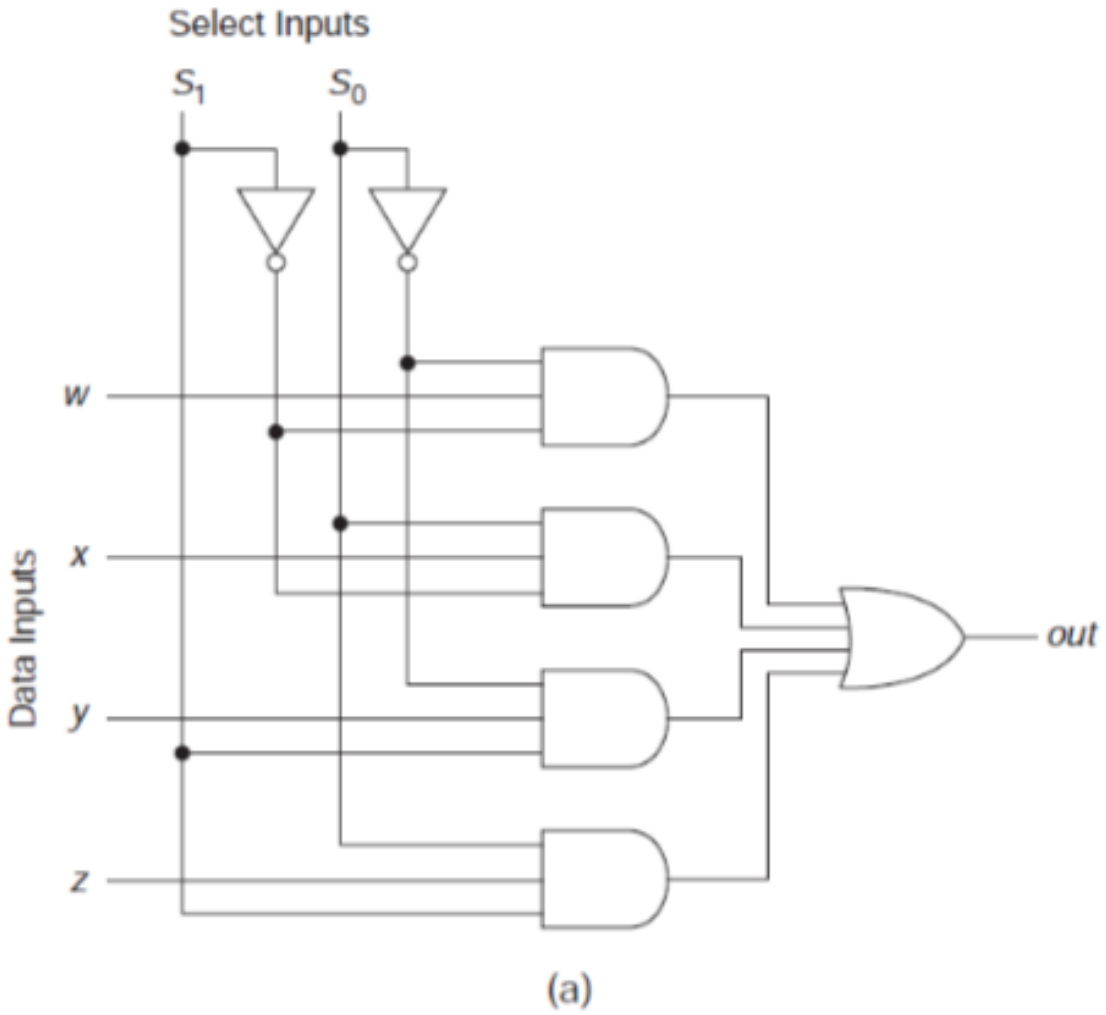
S	out
0	w
1	x

Four-Way Multiplexer

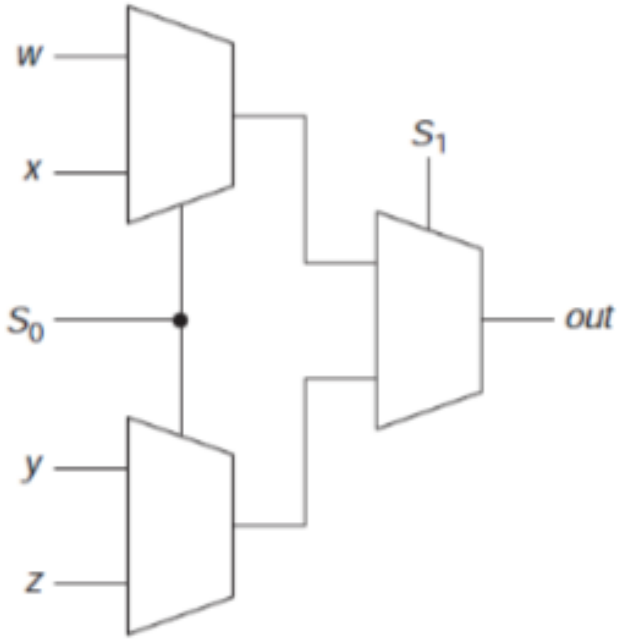
- A four-to-one line multiplexer is shown in Figure 5-12a
- Each of the four inputs is applied to one input of AND gate
- Selection lines S_1 and S_0 are decoded to select a particular AND gate
- Outputs of the AND gates are applied to a single OR gate that provides the one-line output
- A multiplexer is also called a data selector
- – since it selects one of many inputs and steers the binary information to the output line

Four-Way Multiplexer

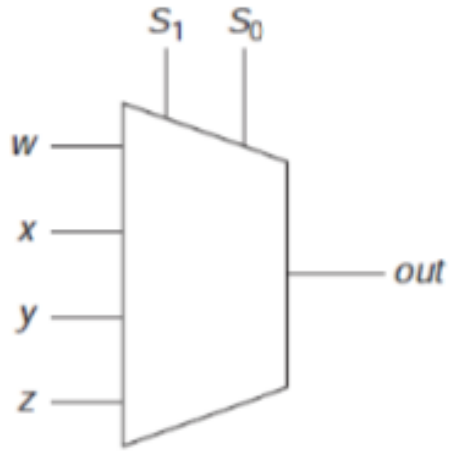
Figure 5.12 (a) A four-way multiplexer. (b) From two-way multiplexers. (c) Logic symbol.



(a)



(b)

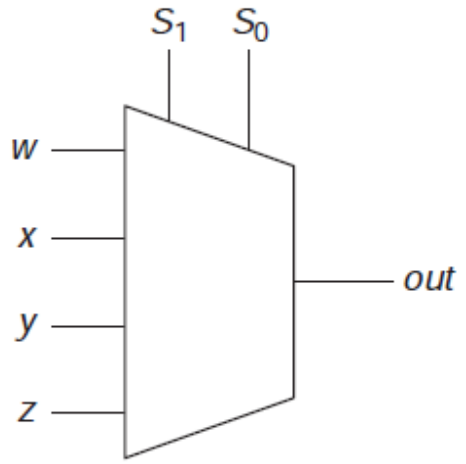


(c)

S_1	S_0	out
0	0	w
0	1	x
1	0	y
1	1	z

Four-Way Multiplexer

- Output equals input
  - *w* if select inputs (S_1, S_2) are 00
  - *x* if they are 01
  - *y* if they are 10
  - *z* if they are 11
- Circuit is very similar to that of decoder, with one AND gate for each select input combination
- Some multiplexers also have *enable* inputs, such that *out* is 0 unless the enable is active



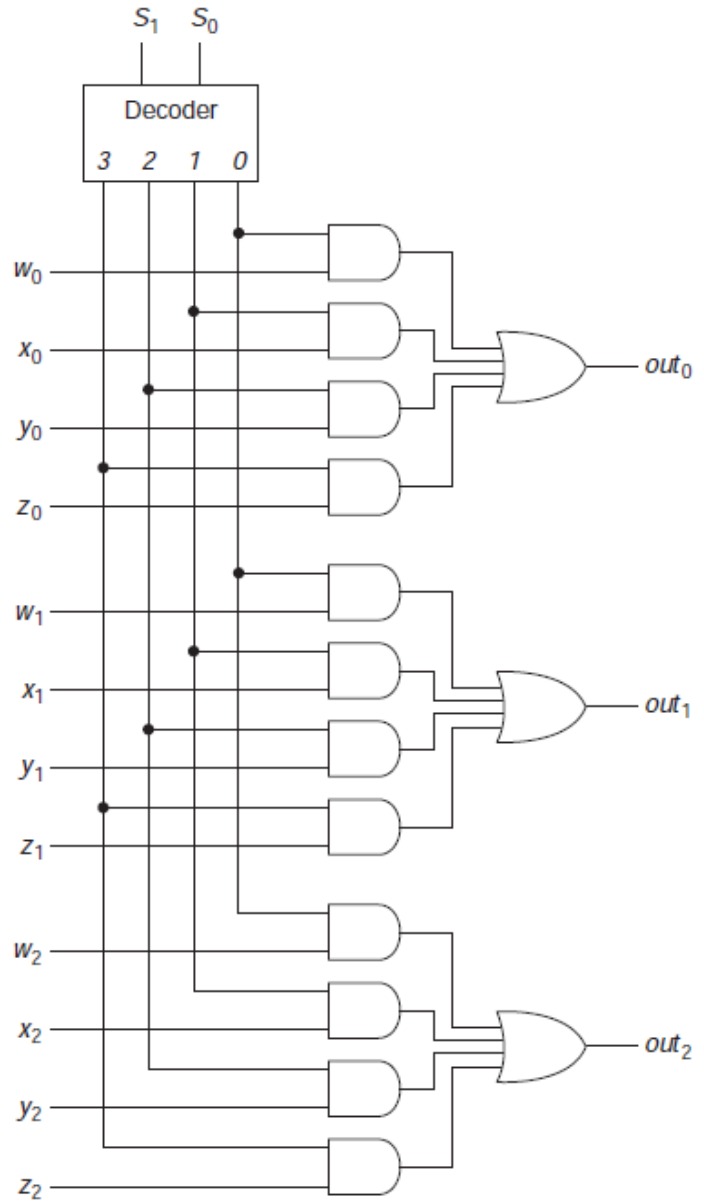
(c)

S_1	S_0	out
0	0	w
0	1	x
1	0	y
1	1	z

Multibit Multiplexer

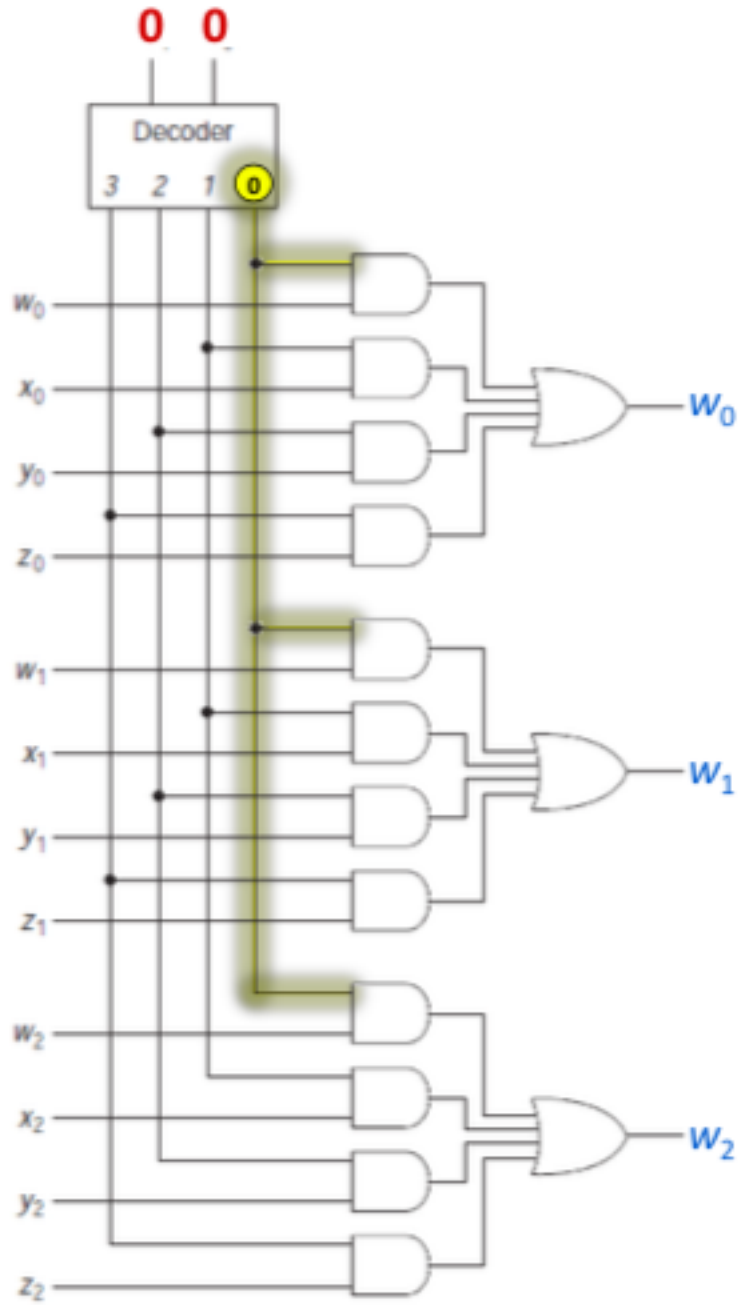
- If inputs consist of set of 16-bit numbers, and control inputs choose which of these numbers is to be passed on, then we would need 16 multiplexers, one for each bit
- We could build 16 of the circuits of Figure 5.12a, utilizing 64 three-input AND gates and 16 four- input OR gates
- Alternative is to use one decoder to drive all the multiplexers
- First 3 bits of such circuit are shown in Figure 5.13

Figure 5.13 A multibit multiplexer.



Multibit Multiplexer Operation

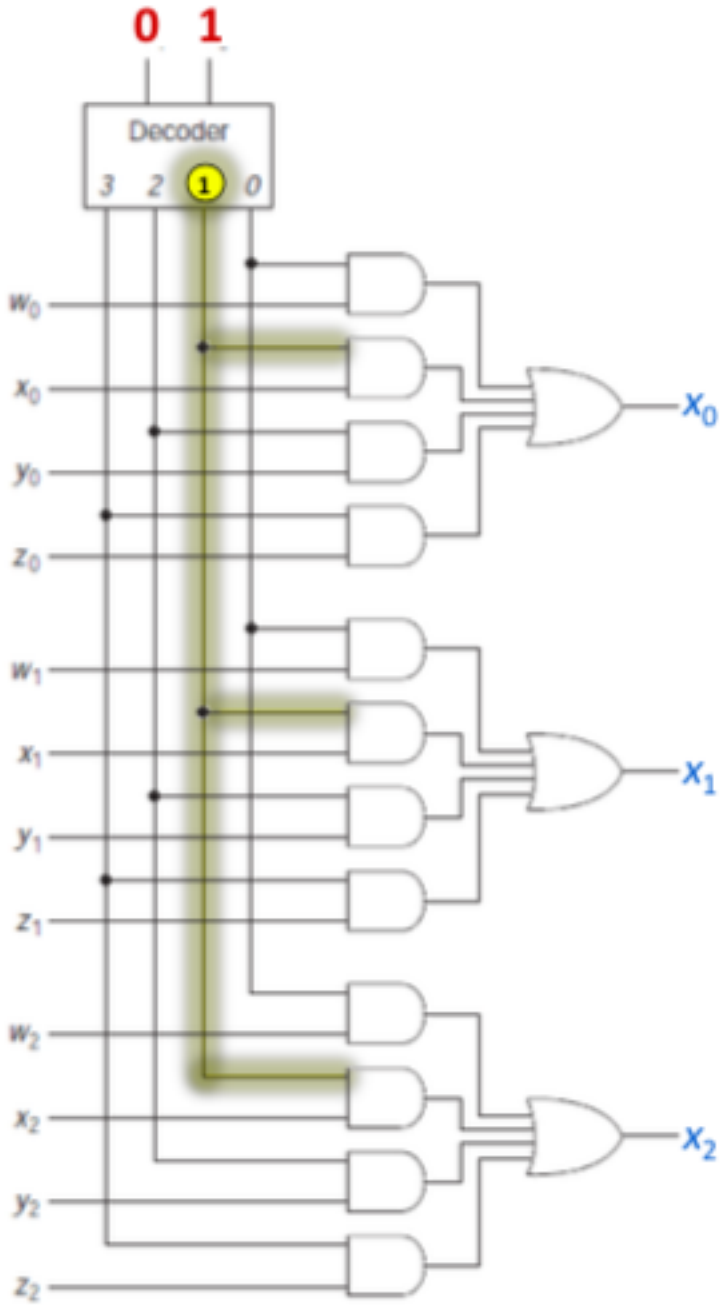
- If *S*0 = 0, *S*1 = 0, *decoder* active output = 0
  - *out*0 = *w*0
  - *out*1 = *w*1
  - *out*2 = *w*2



Multibit Multiplexer Operation

If *S*0 = 0, *S*1 = 1, *decoder* active output = 1

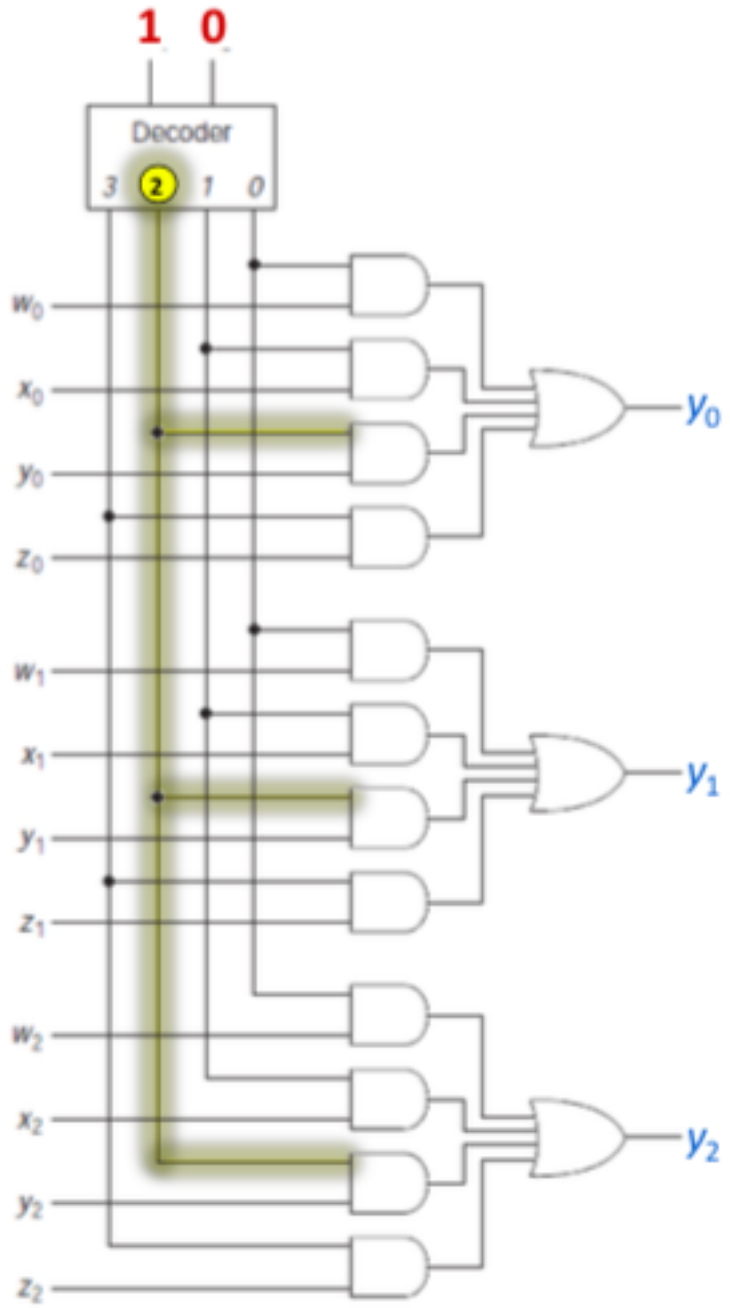
- *out*0 = *x*0
- *out*1 = *x*1
- *out*2 = *x*2



Multibit Multiplexer Operation

If *S*0 = 1, *S*1 = 0, *decoder* active output = 2

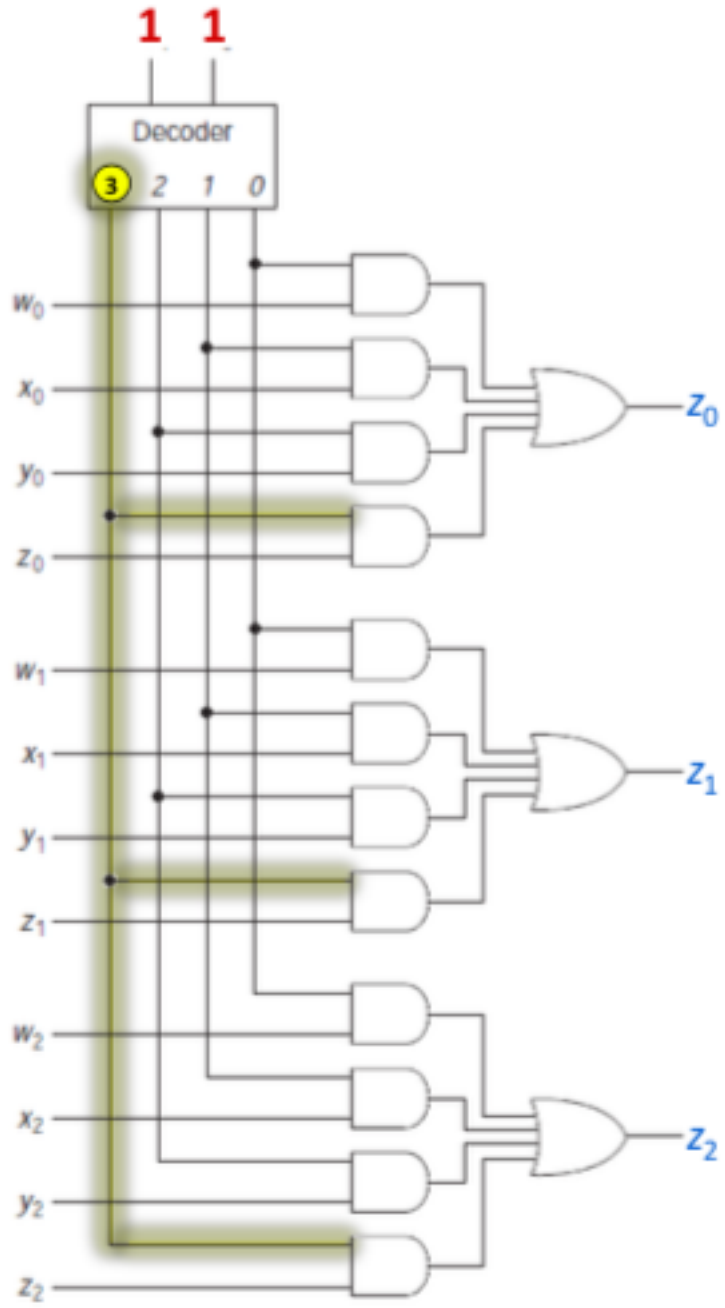
- *out*0 = *y*0
- *out*1 = *y*1
- *out*2 = *y*2



Multibit Multiplexer Operation

If *S*0 = 1, *S*1 = 1, *decoder* active output = 3

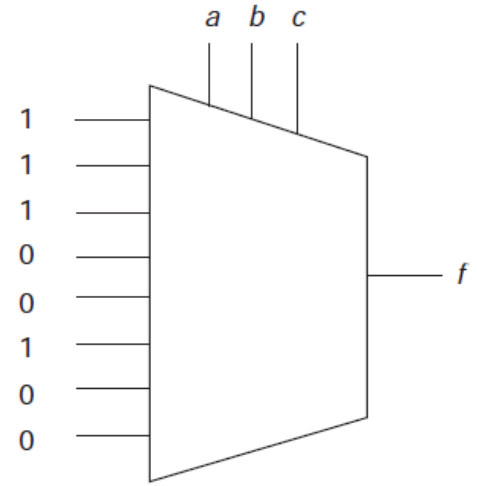
- *out*0 = *z*0
- *out*1 = *z*1
- *out*2 = *z*2



Example

- Consider the Boolean function: $f(a, b, c) = \sum (0, 1, 2, 5)$. TT is as shown.
- It can be implemented with eight-way multiplexer, with values of *f* as input to MUX and variables *a*, *b* and *c* as control inputs (i.e. select lines)
- This MUX is shown as follows:

a *b* *c*	*f*
0 0 0	1
0 0 1	1
0 1 0	1
0 1 1	0
1 0 0	0
1 0 1	1
1 1 0	0
1 1 1	0

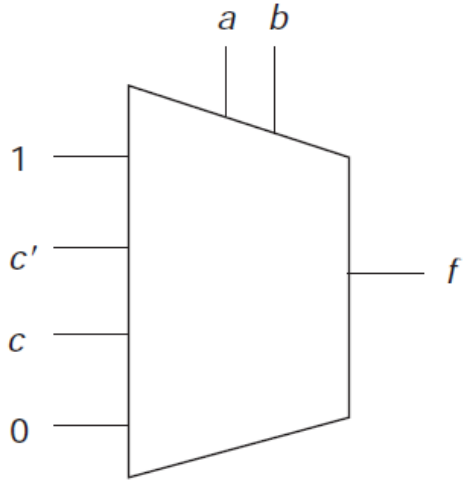


Example cont.

- Same Boolean function can also be implemented with four-way mux
- To do this, we would rewrite truth table as follows; this leads to the shown implementation

a b c	f
0 0 0	1
0 0 1	1
0 1 0	1
0 1 1	0
1 0 0	0
1 0 1	1
1 1 0	0
1 1 1	0

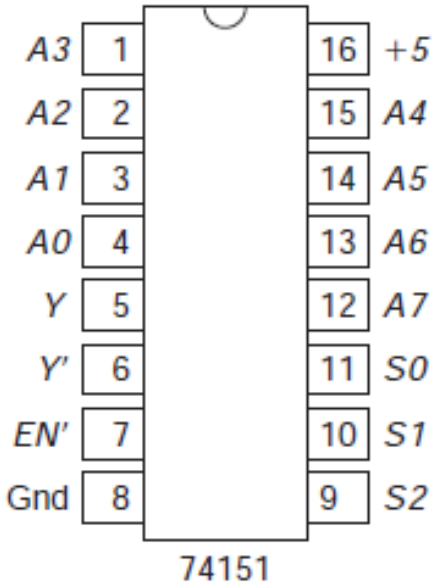
	f		f
a b	c = 0	c = 1	
0 0	1	1	1
0 1	1	0	c'
1 0	0	1	c
1 1	0	0	0



Commercial Multiplexers

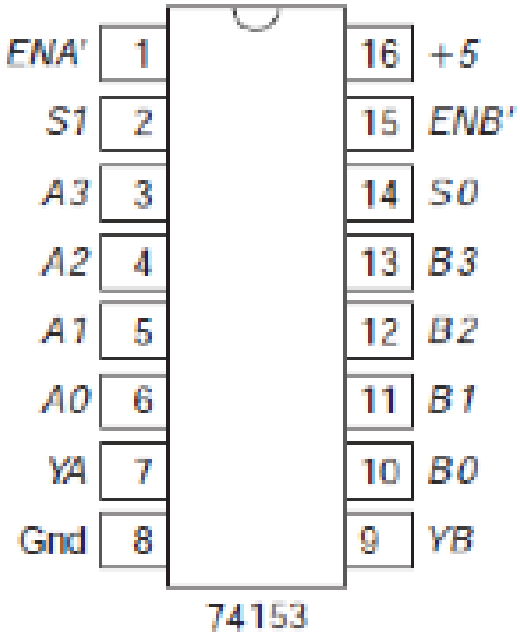
- 74151
  – 1-bit eight-way mux with one active low enable input, *EN'*, and both an uncomplemented (active high) and a complemented (active low) output

  - Inputs are labeled *A7* to *A0* (corresponding to the binary select)
  - Outputs are *Y* and *Y'*
  - Select inputs are labeled *S2*, *S1*, and *S0*, in that order



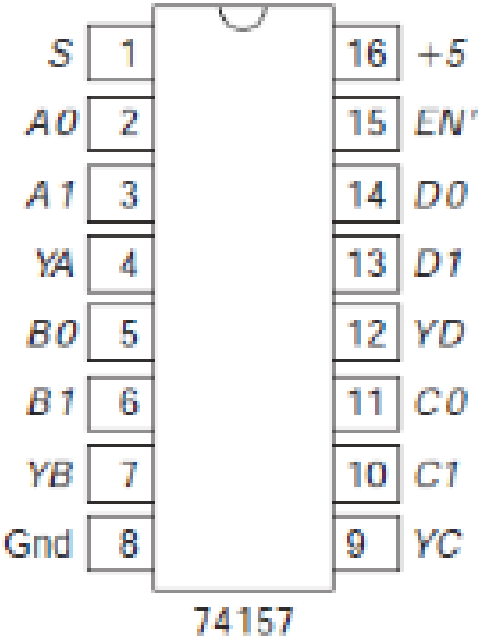
Commercial Multiplexers

- 74153
  – Contains 2 (dual) four-way multiplexers, *A* and *B*, each with its own active low enable (*ENA'* and *ENB'*)
  - Inputs to first are labeled *A3* to *A0* and its output is *YA*
  - Inputs to second are *B3* to *B0*, with output *YB*
  - Two select lines (labeled *S1* and *S0*)
    – Same select signal is used for both multiplexers
    – This would provide 2 bits of multiplexer for choosing among four input words



Commercial Multiplexers

- 74157
  – Contains 4 (quad) two-way multiplexers, with a common active low enable *EN'* and a single common select input *S*; multiplexers are labeled *A*, *B*, *C*, *D*
    - Inputs *A0* and *A1*
    - Output *YA* for the first multiplexer
    - This provides 4 bits of a two-way selection system

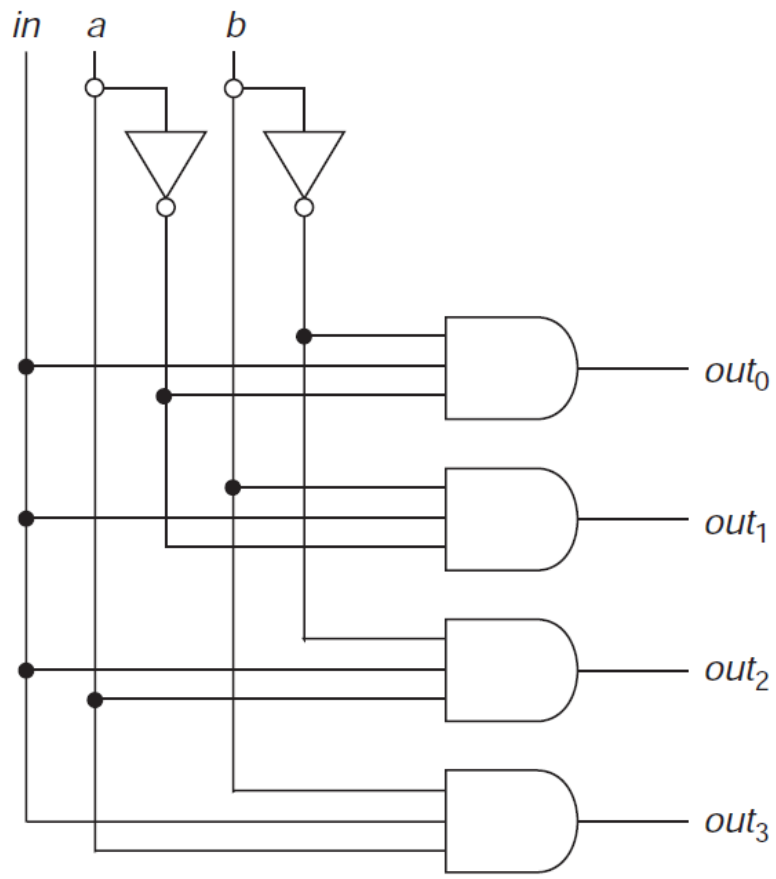


DeMultiplexer

- Demultiplexer (demux) is inverse of mux
  - It routes signal from one place to one of many
  - It is circuit that receives information from single line and directs it to one of 2^n possible outputs
  - Selection of specific output is controlled by bit combination of *n* selection lines

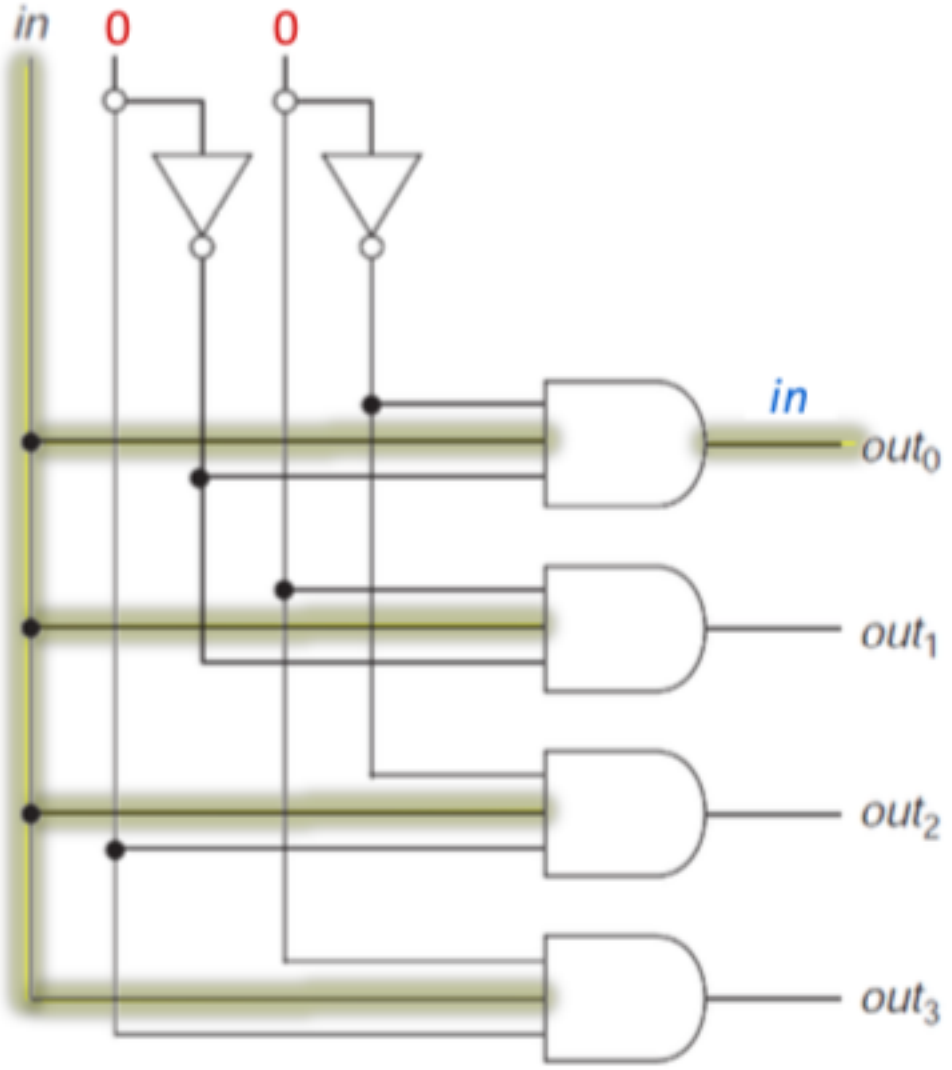
- Figure shows one bit of four-way demux, where *a* and *b* select which way signal, *in*, is directed
  - Circuit is same as for four-way decoder with signal *in* replacing *EN*

Figure 5.14 A four-way demux.



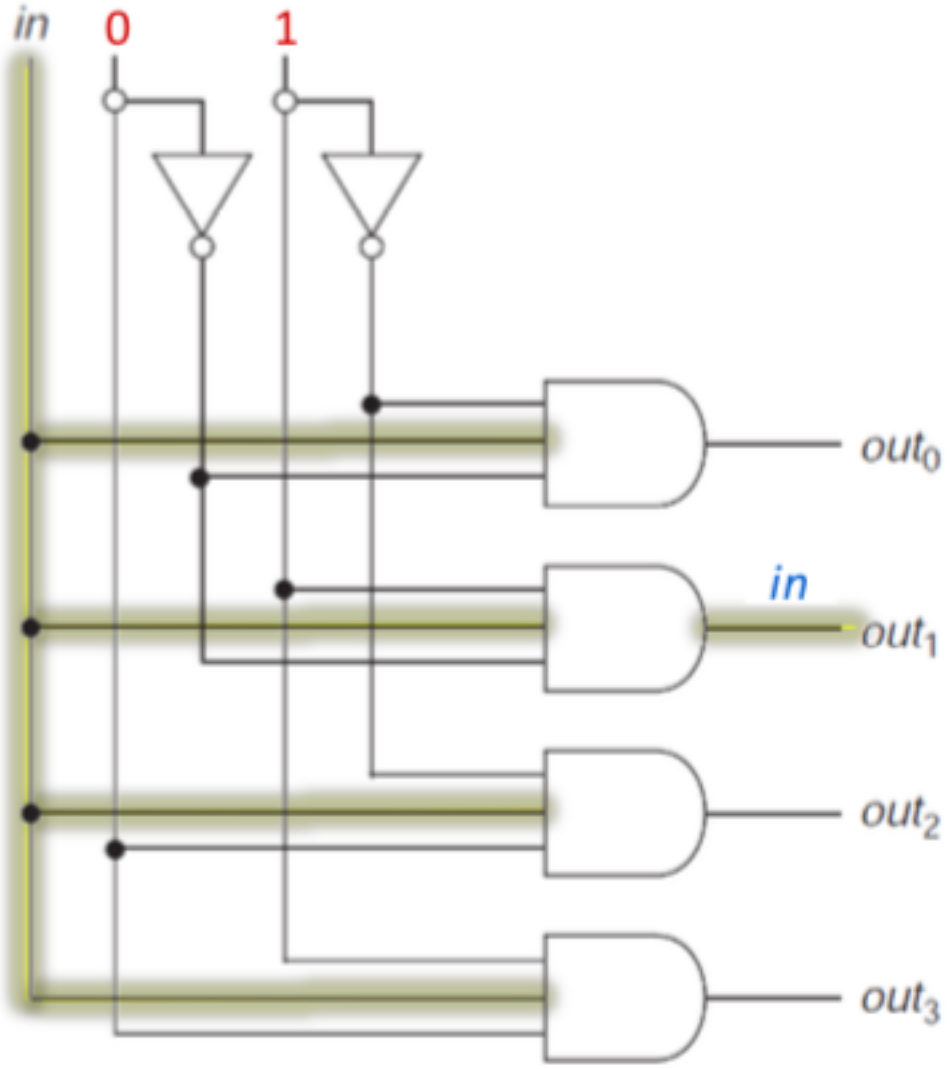
DeMultiplexer Operations

If a = 0, b = 0, *in* input is directed to out_0



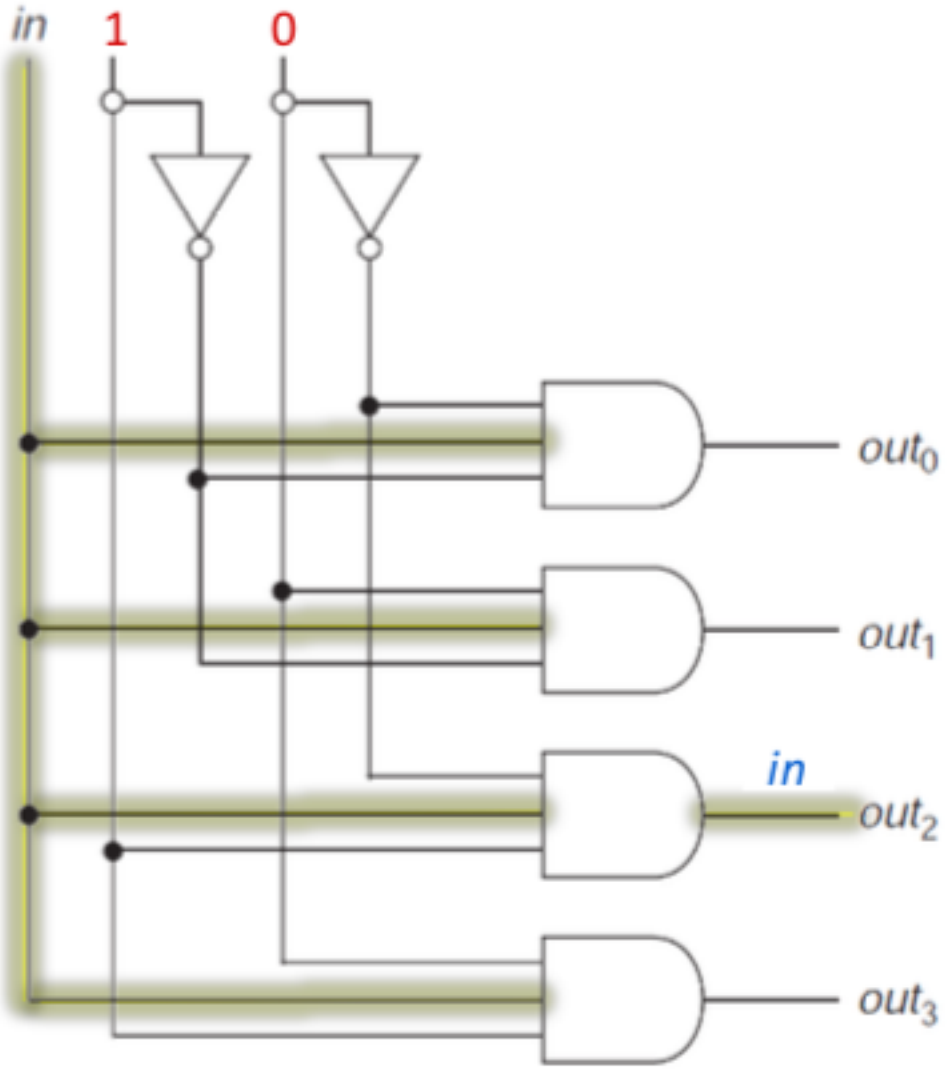
DeMultiplexer Operations

If a = 0, b = 1, *in* input is directed to out_1



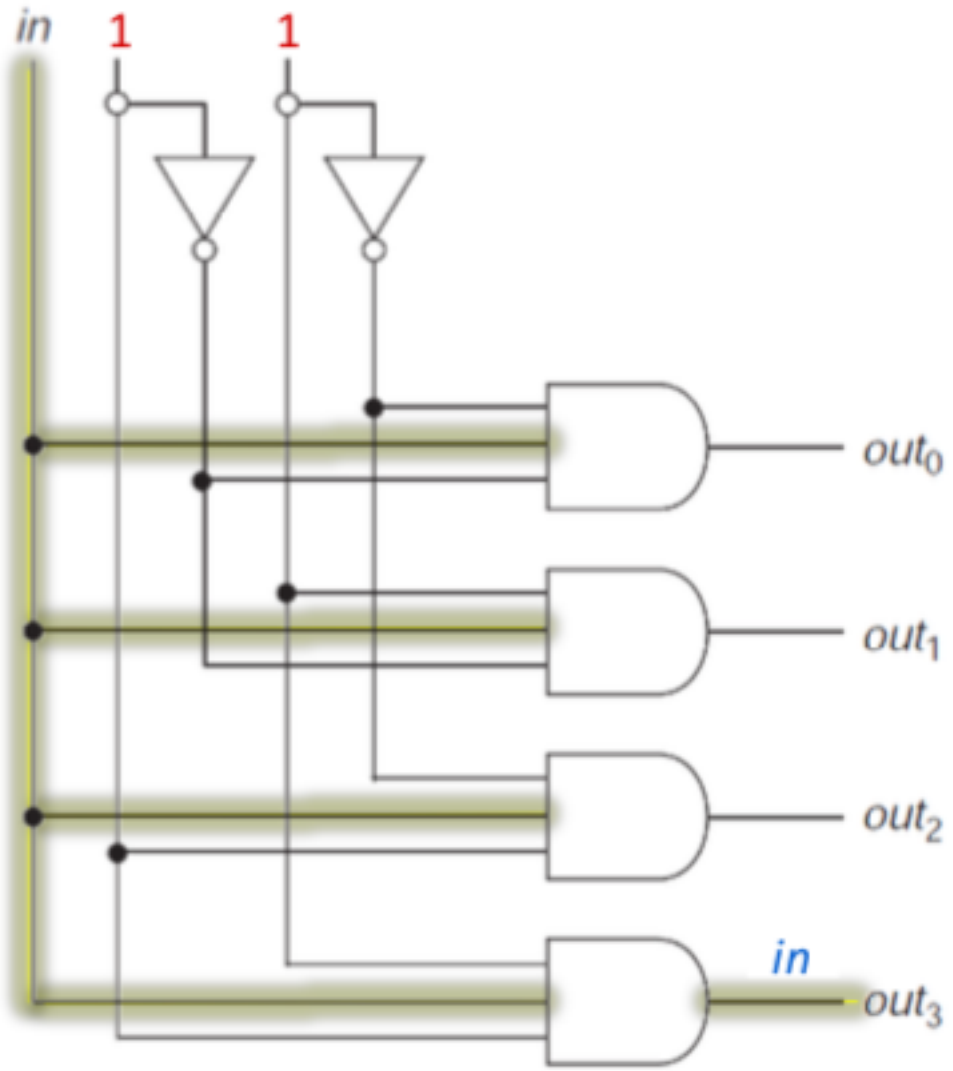
DeMultiplexer Operations

If a = 1, b = 0, *in* input is directed to out_2



DeMultiplexer Operations

If a = 1, b = 1, *in* input is directed to out_3



Three State Gates

- Some design techniques allow us to connect outputs of two gates to each other

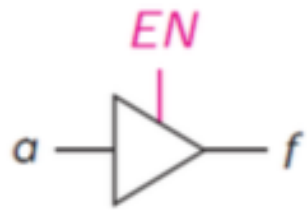
- More commonly used is referred to as three-state (or tristate) output gates

Three State Gates

- In a three-state gate, there is enable *EN* input, shown on side of gate
  - If *EN* is active (active high or active low), gate behaves as usual
  - If *EN* is inactive, output behaves as if it is not connected (open circuit)
    - Output represented by *Z*
- TT and circuit representation of a three-state buffer (with active high enable) is shown

Figure 5.15 A three-state buffer.

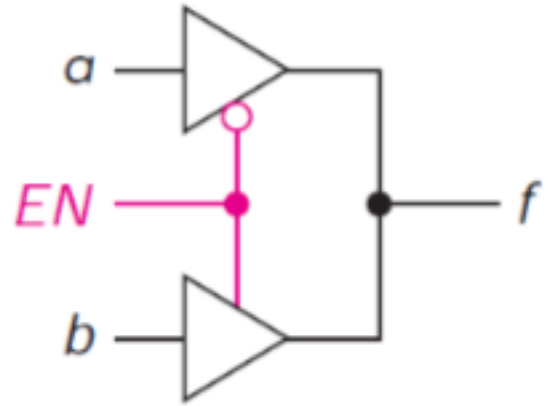
EN	*a*	*f*
0	0	Z
0	1	Z
1	0	0
1	1	1



When ***EN*** = 0 (inactive), output behaves as if it is not connected, represented by ***Z***.

Three State Gates

Figure 5.16 A multiplexer using three-state gates.



- Three-state buffers with active low enables and/or act as a three-state NOT gate
- With three-state gates, we can build a multiplexer without the OR gate
- e.g., circuit of Figure 5.16 is two-way mux
  - *EN* is control input, determining whether
    - *f* = *a* (when *EN* = 0)
    - *f* = *b* (when *EN* = 1)
- Often used for signals that travel between systems

EN	*f*
0	*a*
1	*b*

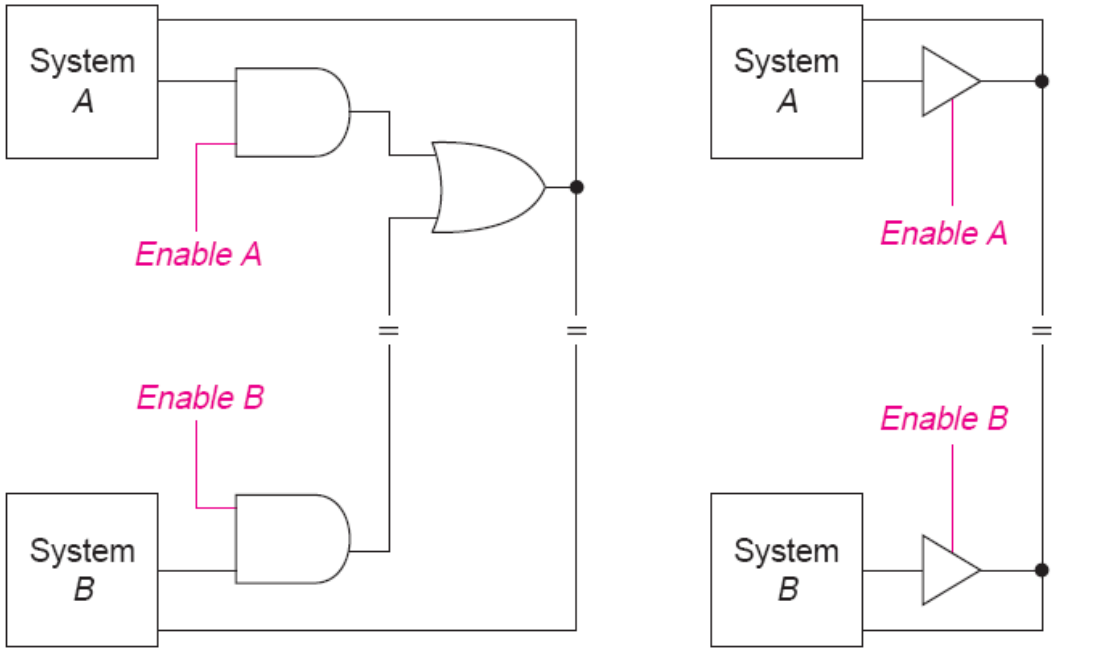
Bus

- A *bus* is a set of lines over which data are transferred
- Sometimes, that data may travel in either direction between devices located physically at a distance
- Bus itself is really just a set of multiplexers, one for each bit in the set

Example

The following circuits show a bit of two implementations of bus using

- AND and OR gates
- three-state gates

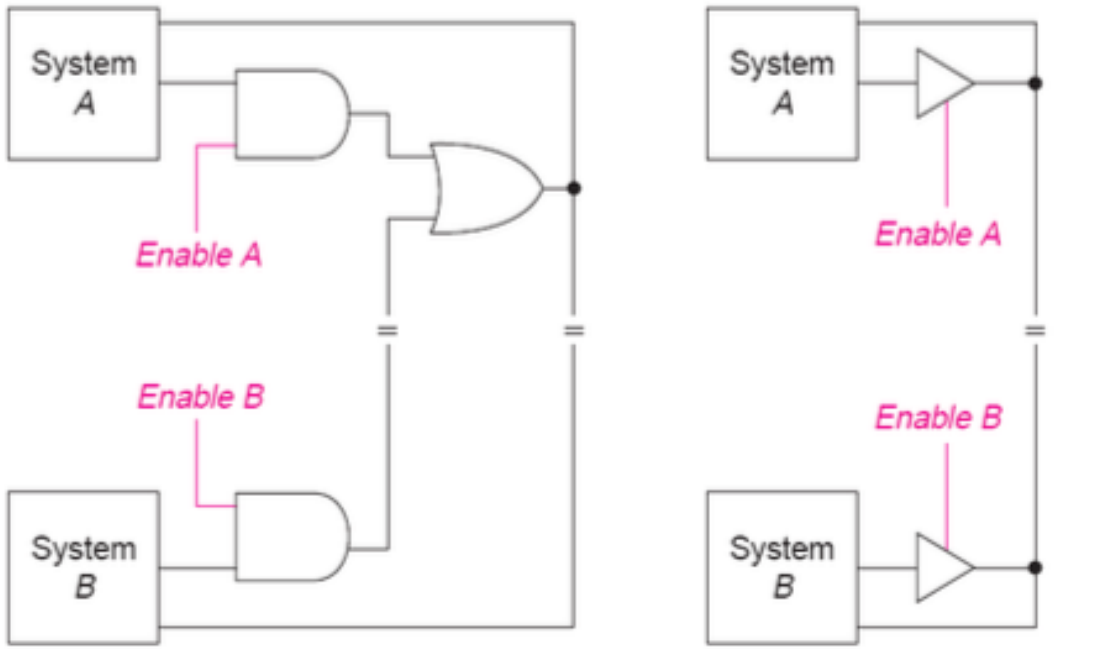


(a) Using AND/OR Gates (b) Using Three-State Gates

Example cont.

Major difference is

– the two long wires per bit between the systems for AND-OR multiplexer compared to only one for the three-state version



(a) Using AND/OR Gates (b) Using Three-State Gates

Example cont.

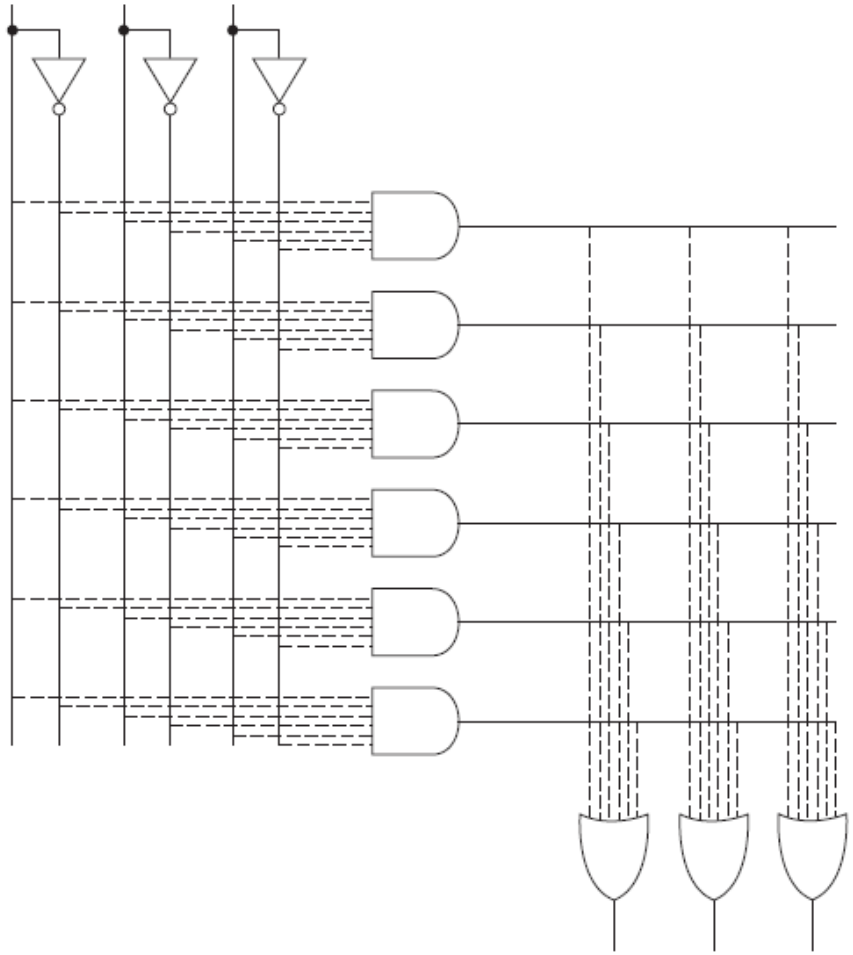
- Difference is even greater if what is being transferred between systems is 32-bit words; in that case, there would be 32 multiplexers
  - AND–OR system would require 64 wires between systems, whereas
  - three-state version would require only 32

- Further, if the enable signal came from one of the systems (instead of being generated internally in each), that would only add one wire, no matter how long the words are

Gate Arrays

- *Gate arrays* are also called *Programmable Logic Devices* (PLDs)
- Gate arrays are one approach to the rapid implementation of fairly complex systems
- They come in several varieties, but all have much in common
- Basic concept is illustrated in Figure 5.17 for system with 3 inputs and 3 outputs where dashed lines indicate possible connections
- These devices implement SOP expressions

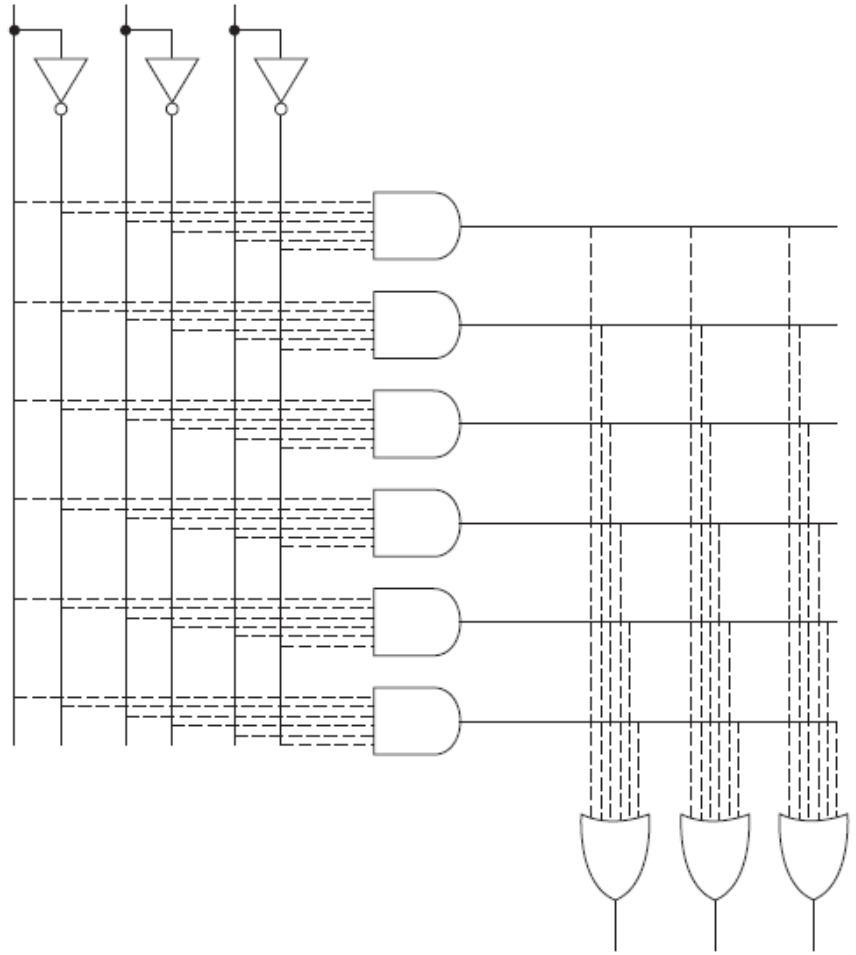
Figure 5.17 Structure of a gate array.



Gate Arrays

- In this case, three functions of three variables can be implemented
- However, since there are only 6 AND gates
  - – a maximum of 6 different product terms can be used for the three functions
- Array only requires uncomplemented inputs

Figure 5.17 Structure of a gate array.

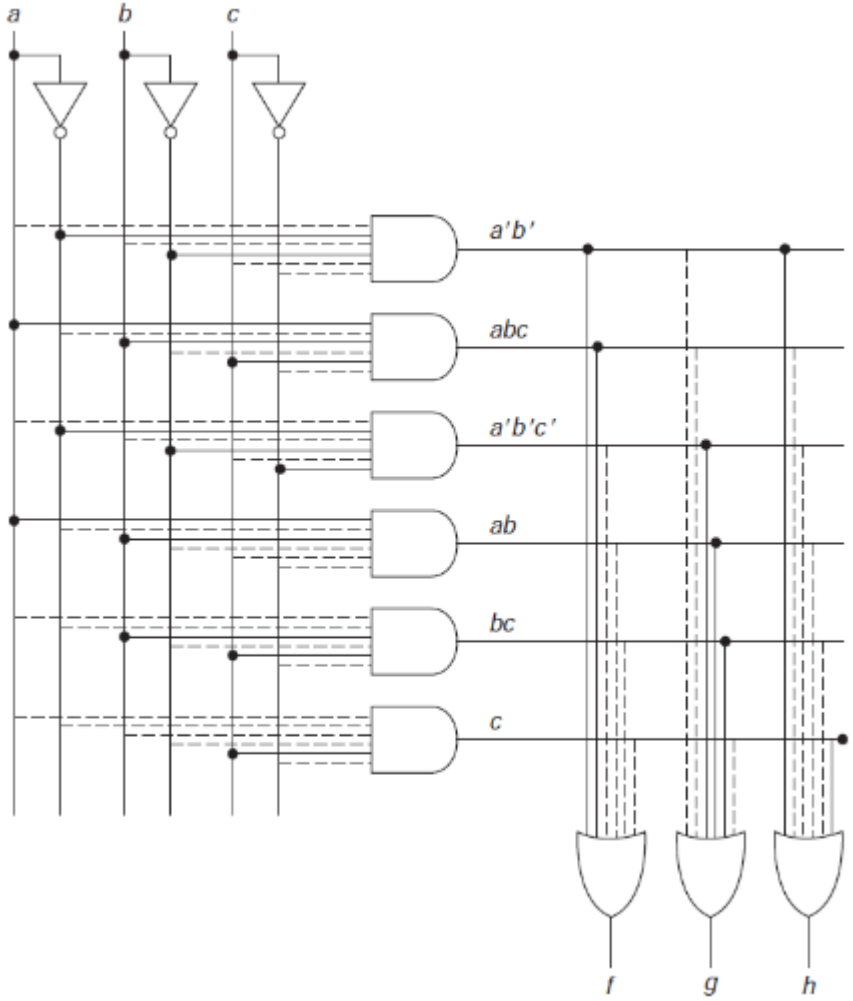


Example

- Consider the following three functions:

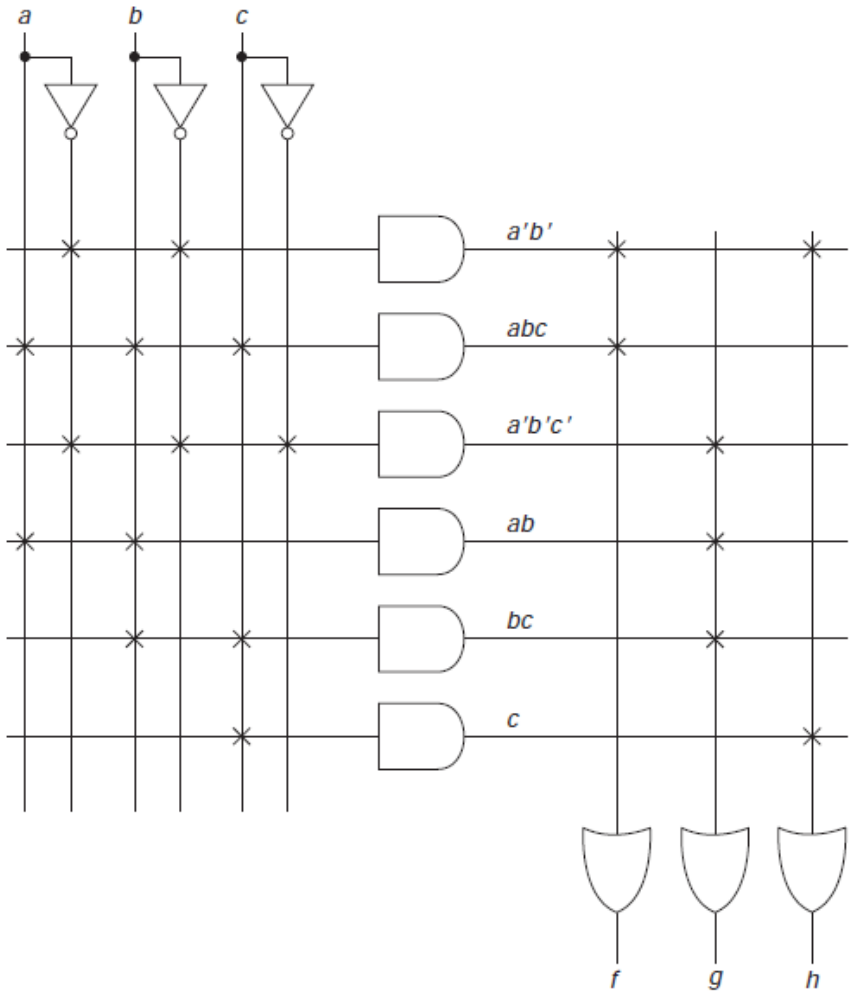
$$f = a'b' + abc$$
$$g = a'b'c' + ab + bc$$
$$h = a'b' + c$$

- These functions can be implemented using the shown gate array
- In this implementation, the solid lines show the actual connections



Example cont.

- Previous version of the diagram is rather cumbersome, particularly as the number of inputs and the number of gates increase
- Thus, rather than showing all of the wires, only a single input line is usually shown for each gate, with X's or dots shown at the intersection where a connection is made
- Previous circuit can be redrawn as shown



Types of Combinational Logic Arrays

Three common types of combinational logic arrays

– Programmable Logic Array (PLA) - the most general type

- User specifies all connections (in both AND array and OR array)
- Can create any set of SOP expressions (and share common terms)

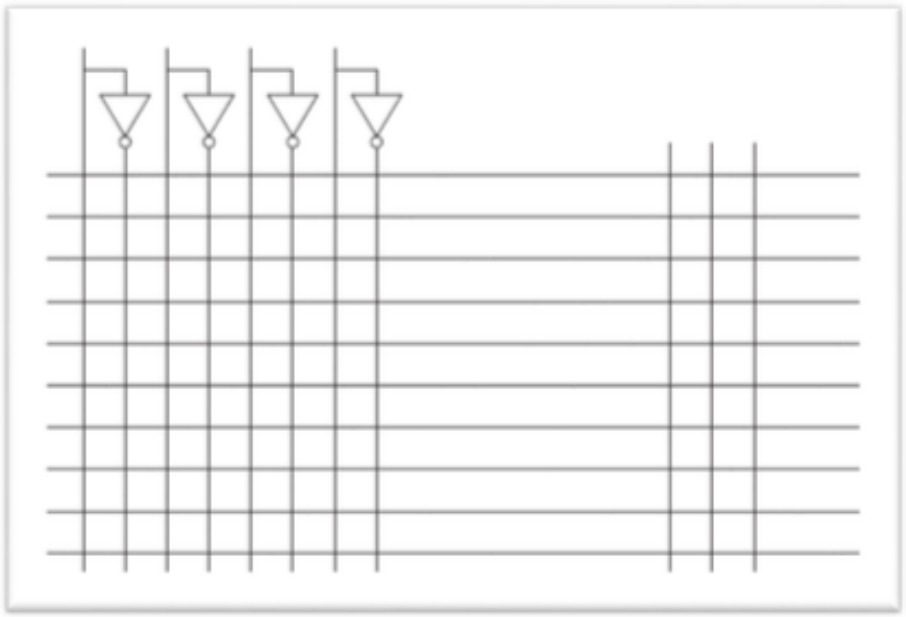
– Read-Only Memory (ROM)

- AND array is fixed
- It is just a decoder, consisting of 2*n* AND gates for an *n*-input ROM
- User only specify connections to OR gate; thus, it produces sum of minterms solution

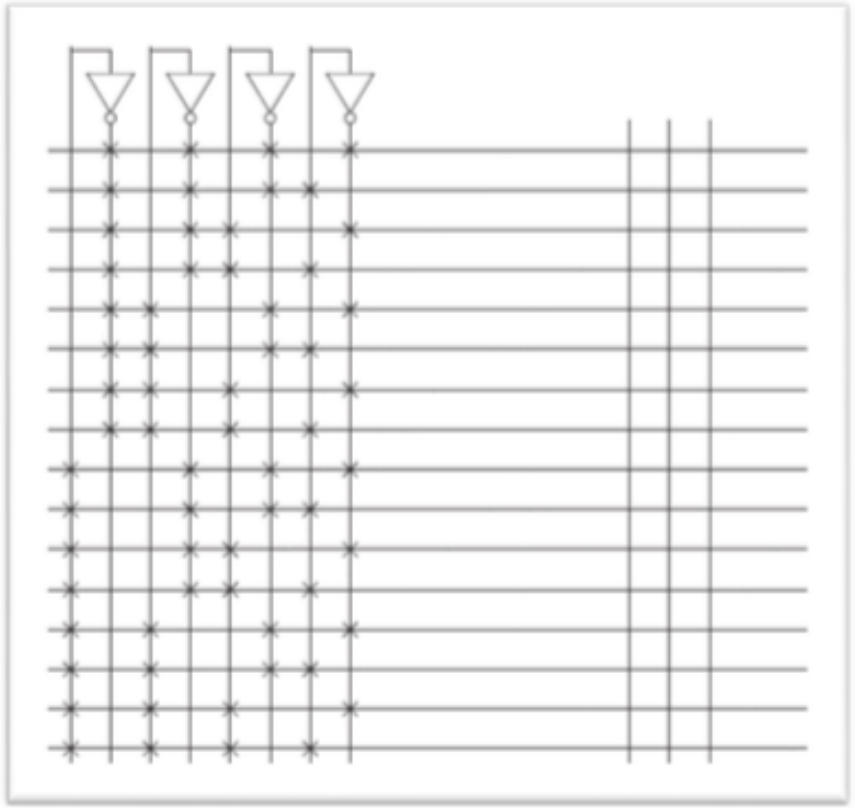
– Programmable Array Logic (PAL)

- Connections to the OR gates are specified
- User can determine the AND gate inputs
- Each product term can be used only for one of the sums

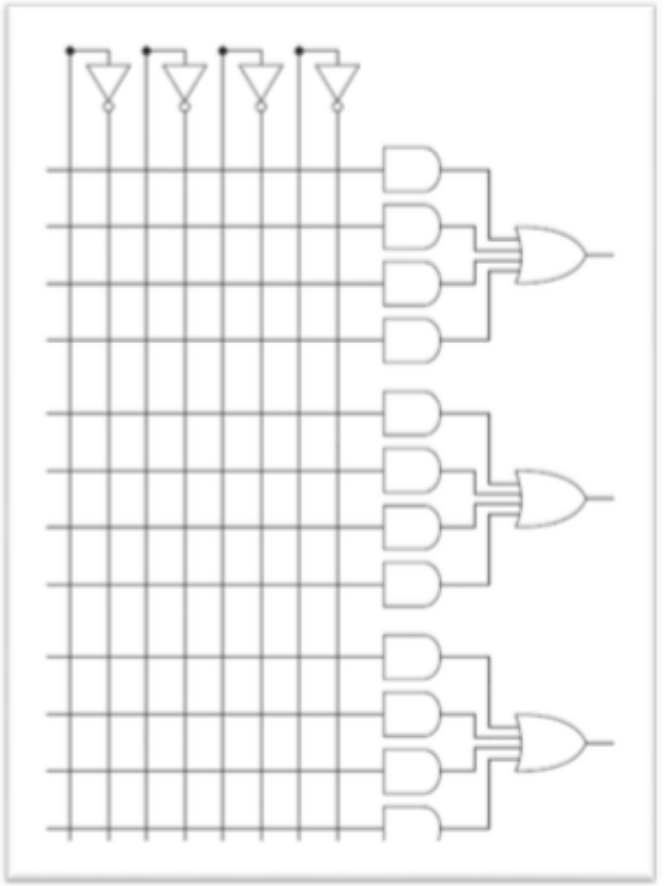
Types of Combinational Logic Arrays



PLA



ROM

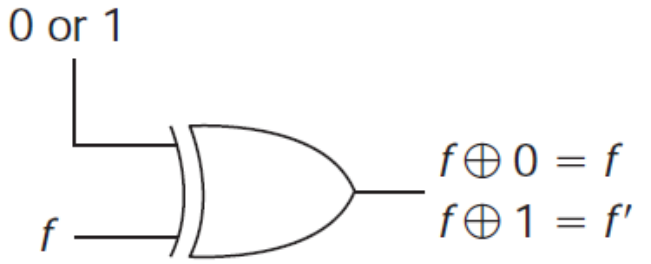


PAL

Types of Combinational Logic Arrays

- In addition to the shown logic, many programmable devices make the output available in either *active high* or *active low* form
  - Active low means complement of output, that is, *f'* instead of *f*
  - This just requires an XOR gate on the output with the ability to program one of the inputs to 0 for *f* and to 1 for *f'*
  - Output logic for such case is shown

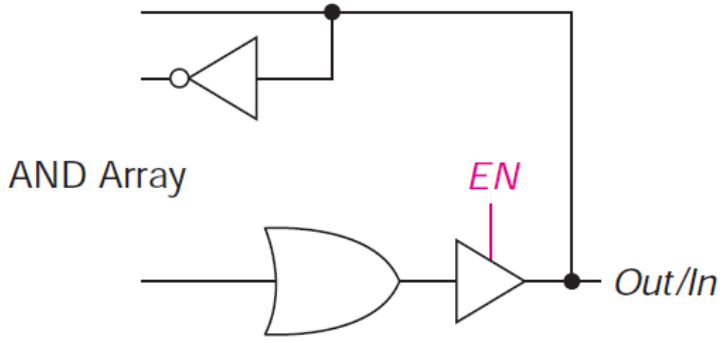
Figure 5.18 A programmable output circuit.



Types of Combinational Logic Arrays

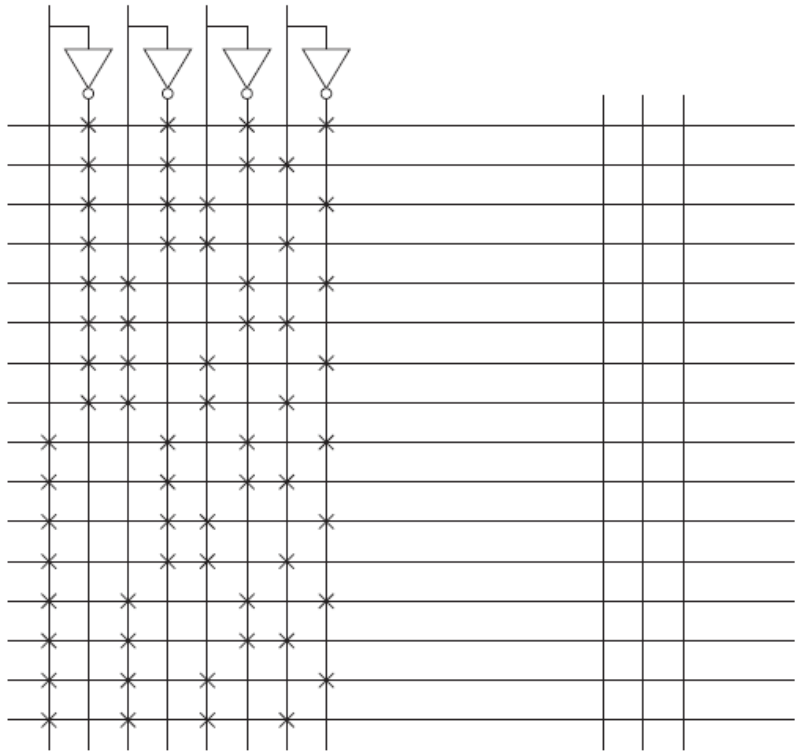
- Some programmable devices have three-state buffer at output, which may be enabled either by an enable input line or one of logic AND gates
  - Allows output to be easily connected to a bus
- Sometimes, output is fed-back as another input to the AND array
  - Allows for more than two-level logic (most commonly in PALs)
  - Also allows that output to be used as input instead of output, if three-state output gate is added, as shown

Figure 5.19 Three-state output.



Designing with ROMs

- To design a system using a ROM, function should be represented in minterms
- ROM has one AND gate for each minterm
  - Conect the appropriate minterm gates to each output
- This is really the same type of circuitry as the decoder implementation of a sum of product expression



Example

Implement the following three functions with ROM

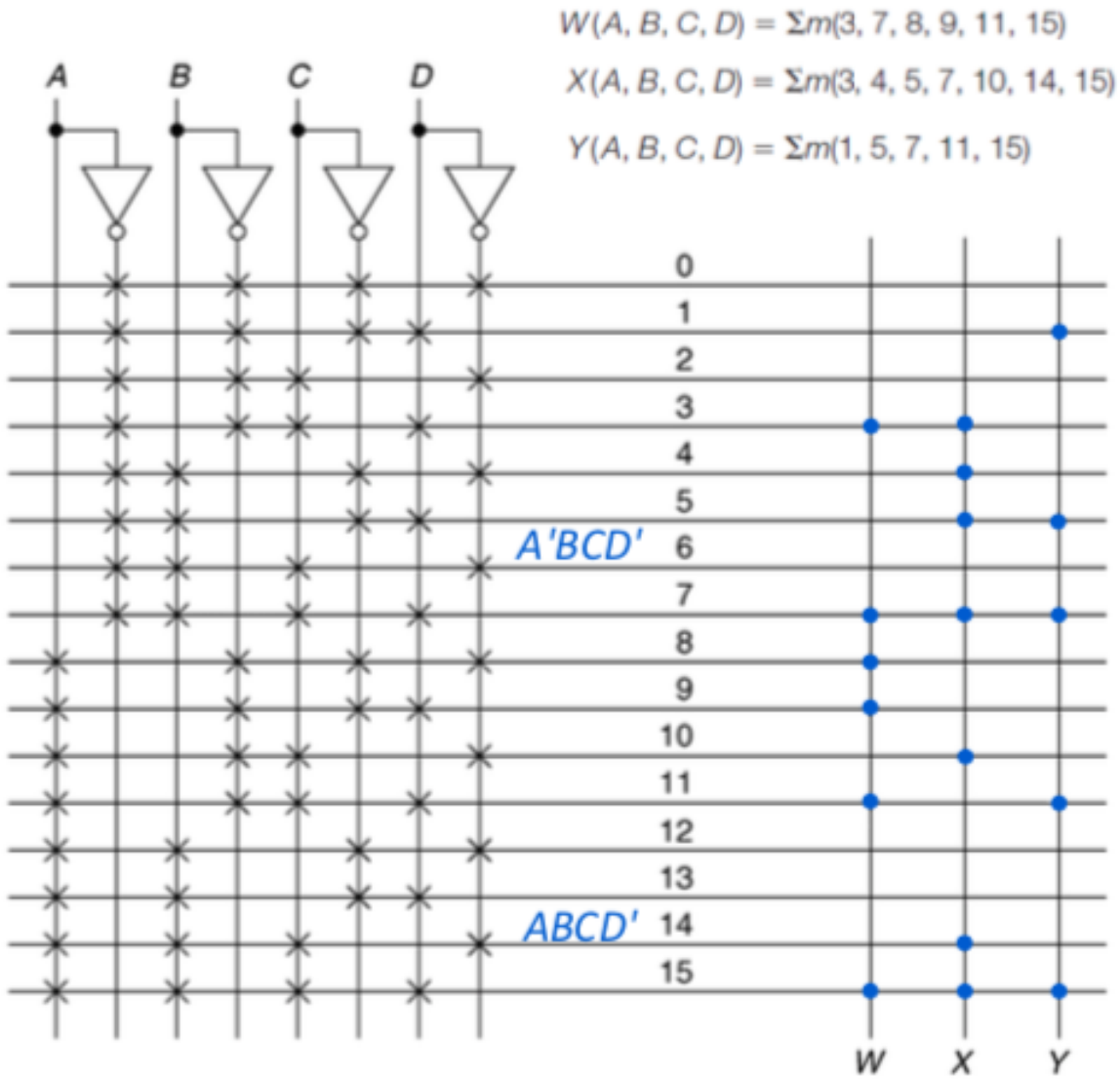
$$W(A, B, C, D) = \Sigma m(3, 7, 8, 9, 11, 15)$$
$$X(A, B, C, D) = \Sigma m(3, 4, 5, 7, 10, 14, 15)$$
$$Y(A, B, C, D) = \Sigma m(1, 5, 7, 11, 15)$$

- Since, there are four inputs *A*, *B*, *C* and *D*
- Rows of the ROM are numbered (in order) from 0 to 15.
- An X or a dot ● is then placed at the appropriate intersection

Example cont.

- For example, for minterm
  - 6, X's are placed at *A'BCD'*
  - 14, X's are placed at *ABCD'*

- To implement given functions, pass the corresponding minterms by placing dots at intersection of required minterms, giving outputs *W*, *X* and *Y*



Designing with Programmable Logic Arrays

- To design a system using a PLA, you need only find SOP (instead of Sum of Minterms as in case of designing with ROMs) expressions for the functions to be implemented
- The only limitation is the number of AND gates (product terms) that are available
- Any SOP expression for each of functions will be obtained by simplifying functions using K-Maps so that we can maximize sharing of gates

Example

Implement the following three functions with PLA

$$W(A, B, C, D) = \Sigma m(3, 7, 8, 9, 11, 15)$$
$$X(A, B, C, D) = \Sigma m(3, 4, 5, 7, 10, 14, 15)$$
$$Y(A, B, C, D) = \Sigma m(1, 5, 7, 11, 15)$$

- In order to implement these functions with PLA, we shall simplify them first by K-maps and then try to combine 1's such that we can maximize sharing of terms

Example cont.

- So, K-maps for w, x and y are as follows:
- Implementation of functions with PLA is shown

$W(A, B, C, D) = \Sigma m(3, 7, 8, 9, 11, 15)$

CD \ AB	00	01	11	10
00				1
01				1
11	1	1	1	1
10				

W

$X(A, B, C, D) = \Sigma m(3, 4, 5, 7, 10, 14, 15)$

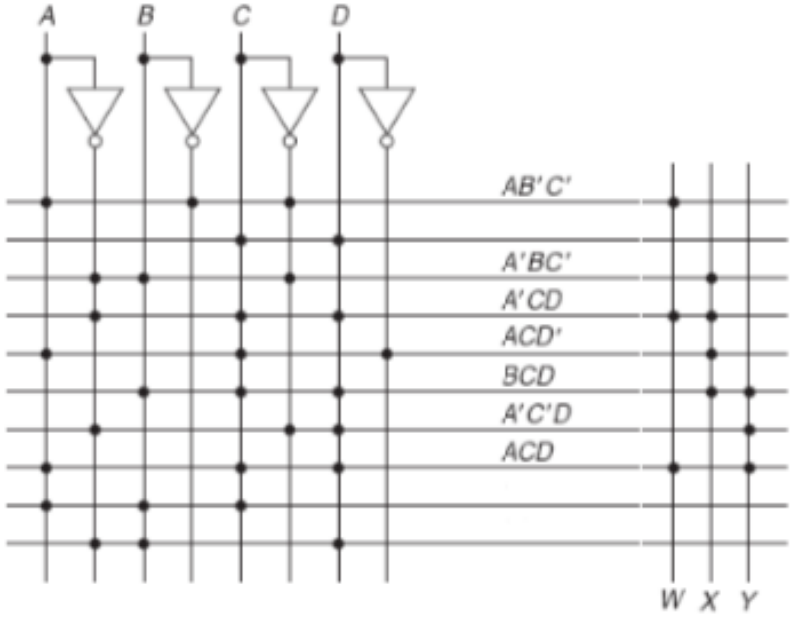
CD \ AB	00	01	11	10
00		1		
01		1		
11	1	1	1	
10			1	1

X

$Y(A, B, C, D) = \Sigma m(1, 5, 7, 11, 15)$

CD \ AB	00	01	11	10
00				
01	1	1		
11		1	1	1
10				

Y



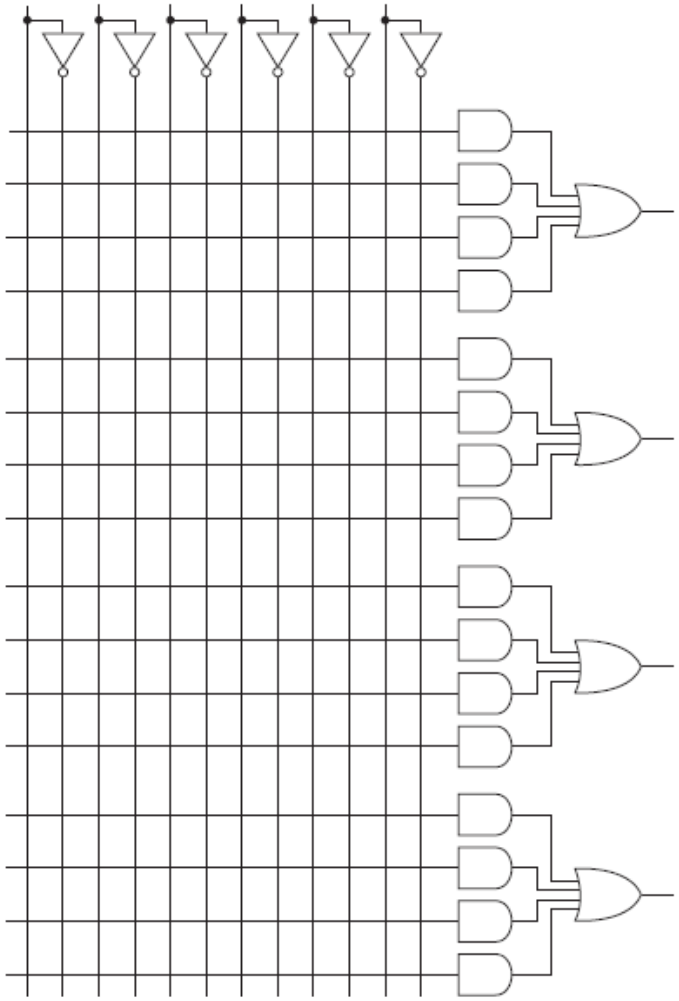
$W = AB'C' + A'CD + ACD$

$X = A'BC' + A'CD + BCD + ACD'$

$Y = A'C'D + BCD + ACD$

Designing with Programmable Array Logic

- In PAL, each output comes from an OR gate that has its own group of AND gates connected to it
- Layout of a small PAL is shown
  - For this PAL, there are six inputs and four outputs, with each OR gate having four input terms
- When using PAL, output of each AND gate goes to only one OR
  - No sharing of terms, solve each function individually
  - However, most PALs provide for possible feedback of some or all of outputs to input



Example

Implement the following three functions with PAL.

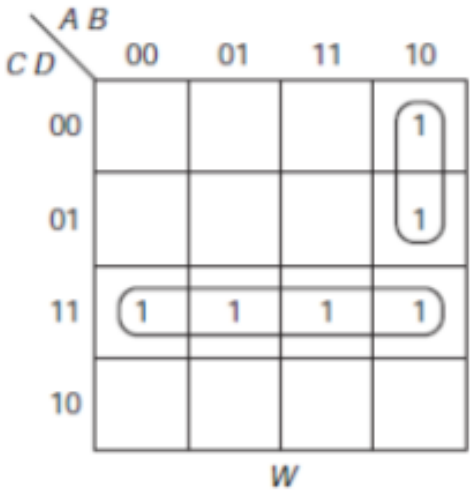
$$W(A, B, C, D) = \Sigma m(3, 7, 8, 9, 11, 15)$$
$$X(A, B, C, D) = \Sigma m(3, 4, 5, 7, 10, 14, 15)$$
$$Y(A, B, C, D) = \Sigma m(1, 5, 7, 11, 15)$$

- In order to implement these functions with PAL, we shall simplify them first by using K-maps and then try to combine 1's such that we can minimize the functions without considering the sharing of terms
- So, K-maps for w, x and y are as follows:

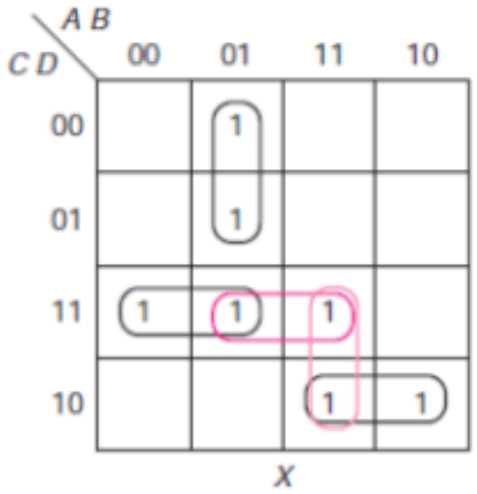
Example cont.

- Implementation of the given functions with PAL (choosing first of each of optional terms) is as follows:

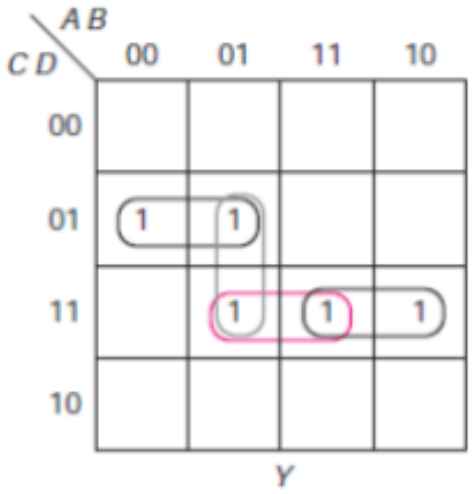
$W(A, B, C, D) = \Sigma m(3, 7, 8, 9, 11, 15)$



$X(A, B, C, D) = \Sigma m(3, 4, 5, 7, 10, 14, 15)$



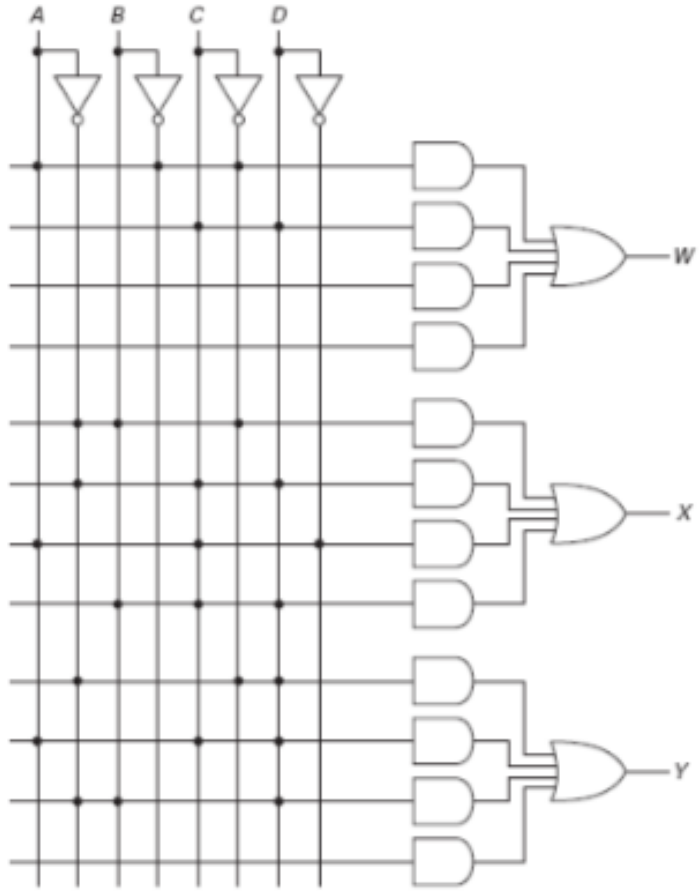
$Y(A, B, C, D) = \Sigma m(1, 5, 7, 11, 15)$



$W = AB'C' + CD$

$X = A'BC' + A'CD + ACD' + \{ BCD + ABC \}$

$Y = A'C'D + ACD + \{ A'BD + BCD \}$



Example

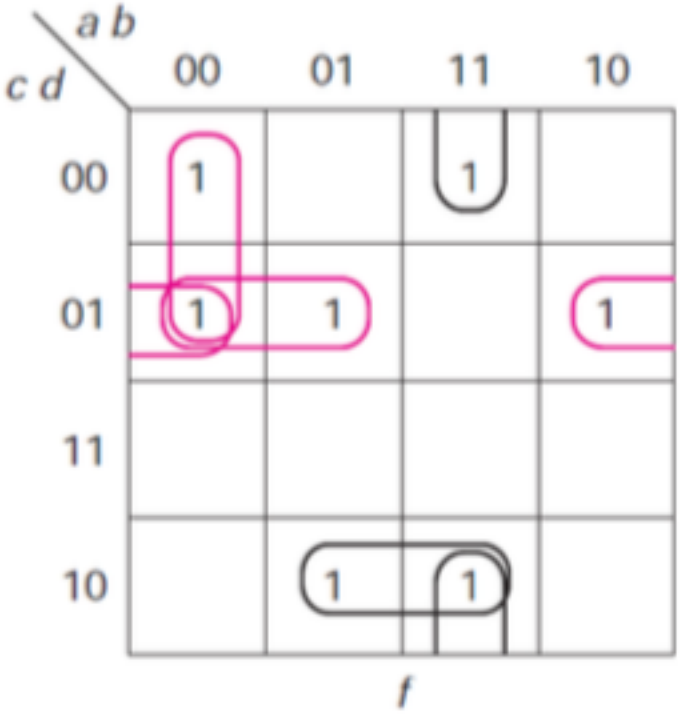
- Implement the following three functions with PAL.

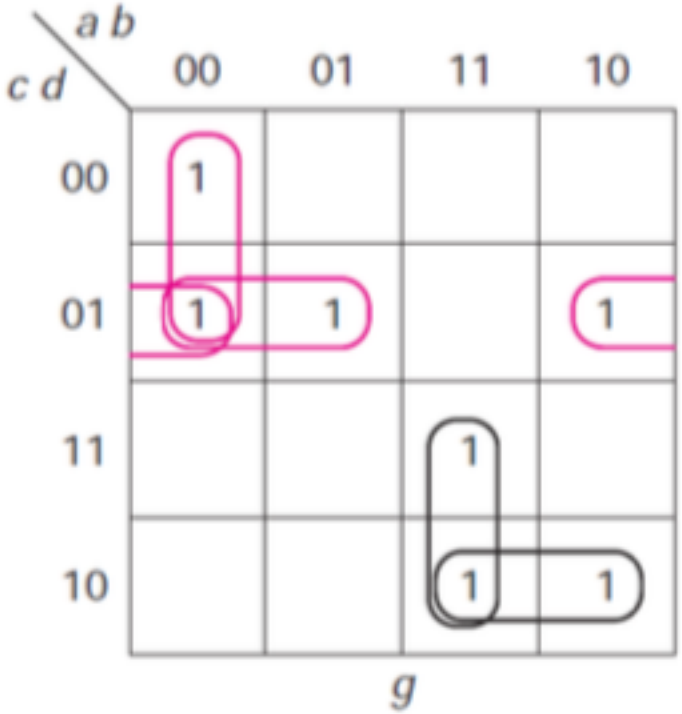
$f(a, b, c, d) = \sum(0, 1, 5, 6, 9, 12, 14)$
$g(a, b, c, d) = \sum(0, 1, 5, 9, 10, 14, 15)$
$h(a, b, c, d) = \sum(0, 1, 2, 5, 6, 9, 10)$

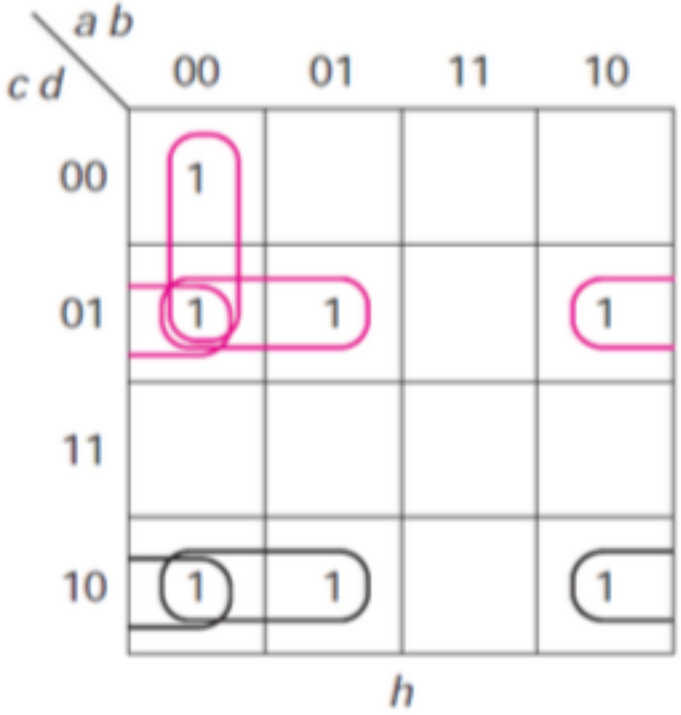
- In order to implement these functions with PAL, we shall simplify them first by using K-maps and then try to combine 1's such that we can minimize the functions without considering the sharing of terms.

- So, K-maps for *f*, *g* and *h* are as follows:

Example cont.







$$f = a'b'c' + a'c'd + b'c'd + abd' + bcd'$$

$$g = a'b'c' + a'c'd + b'c'd + abc + acd'$$

$$h = a'b'c' + a'c'd + b'c'd + a'cd' + b'cd'$$

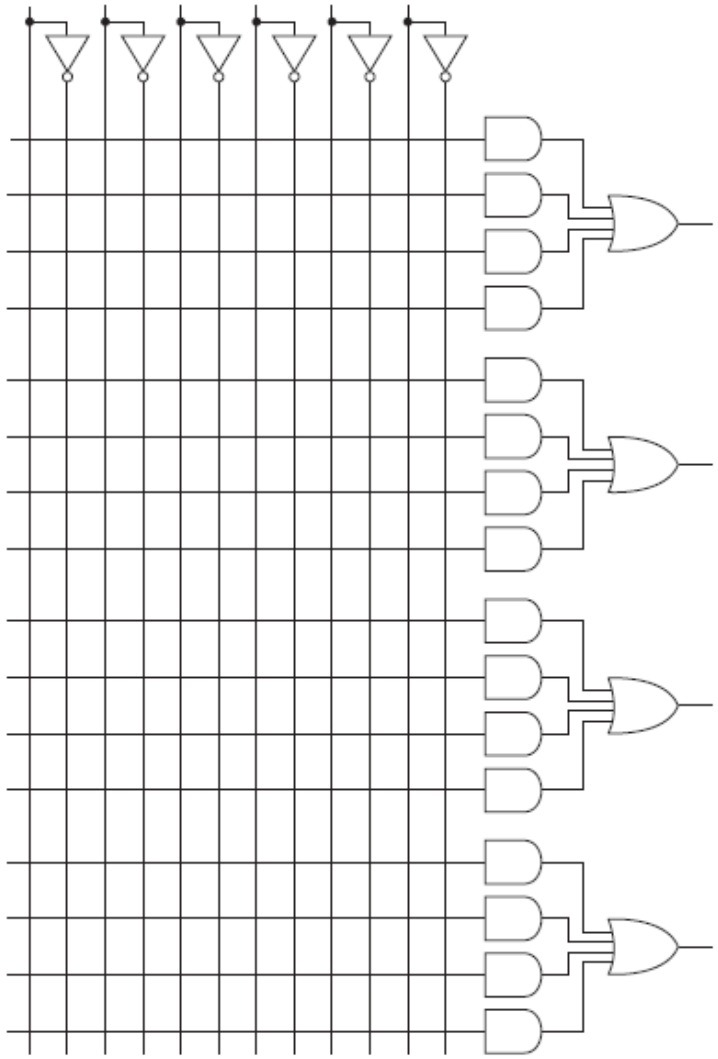
Example cont.

- Simplified functions contain more than four terms
- Therefore, cannot be implemented with PAL
- Usefeedbacksolution

$$f = a'b'c' + a'c'd + b'c'd + abd' + bcd'$$

$$g = a'b'c' + a'c'd + b'c'd + abc + acd'$$

$$h = a'b'c' + a'c'd + b'c'd + a'cd' + b'cd'$$



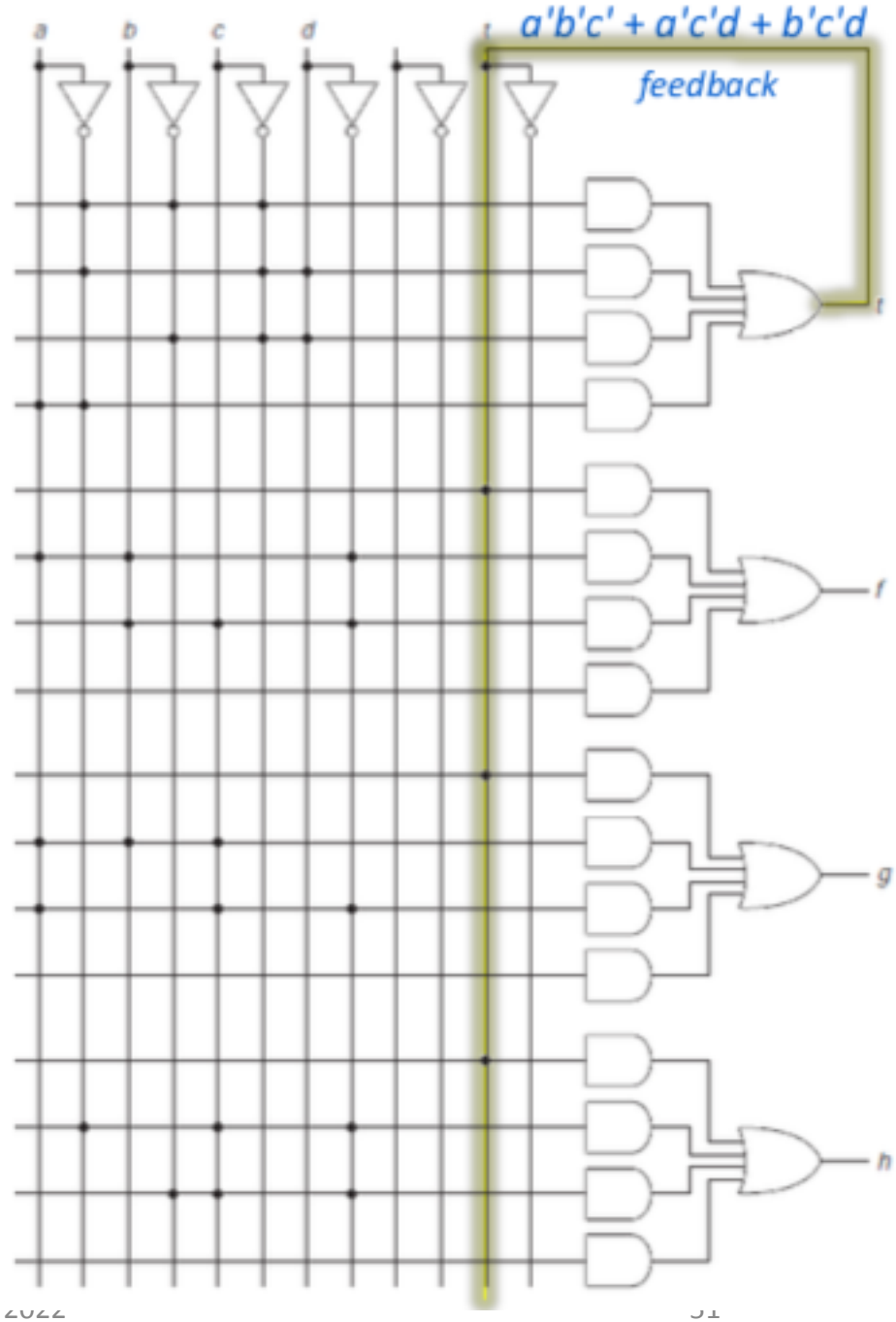
Example cont.

• So, first three common terms (prime implicants) are implemented in first OR gate, output of which, *t*, is *fed-back* to the input of one of the AND gates in each of the other three circuits

• Note that the fourth AND gate of *t* circuit has both *a* and *a'* connected at its input because output of that AND gate is 0

– Some implementations require the user to connect unused AND gates in that way

– Not done for other unused AND gates




The End